



第10课时：循环控制

break 与 continue

跳出循环 · 跳过迭代 · 嵌套循环入门



为什么需要循环控制？



循环默认会一直执行直到条件为假，但有时我们需要：

- **提前终止循环**（找到目标后不再继续）
- **跳过某次迭代中的剩余操作**（但不终止整个循环）
- 在**嵌套循环**中灵活控制内层或外层

C++ 提供了 `break` 和 `continue` 两个关键字来实现这些需求。

break 语句



作用：立即**终止**当前所在的最内层循环，程序跳转到循环之后的代码。

```
while (条件) {  
    if (某个条件) {  
        break;    // 满足条件时跳出循环  
    }  
    // 剩余代码  
}  
// 执行到这里
```

适用场景：

- 搜索元素，找到后立即停止
- 猜数字游戏，猜中后退出
- 输入验证，遇到特定值终止

break 示例：查找第一个偶数



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int arr[] = {3, 7, 9, 2, 8, 1};
    int n = 6;
    int pos = -1;
    for (int i = 0; i < n; i++) {
        if (arr[i] % 2 == 0) {
            pos = i;
            break;           // 找到第一个偶数，立即退出
        }
    }
    cout << "第一个偶数的下标是: " << pos << endl;
    return 0;
}
// 输出：第一个偶数的下标是：3
```

continue 语句



作用：跳过当前迭代中 `continue` 之后的剩余语句，直接进入下一次循环的判断（更新 → 条件检查）。

```
while (条件) {  
    if (某个条件) {  
        continue;    // 跳过本次循环剩余部分  
    }  
    // 这行如果被跳过则不会执行  
}
```

适用场景：

- 过滤数据：只处理满足特定条件的元素
- 忽略某些特殊情况（如输入为负数时不处理）

continue 示例：只输出奇数



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;    // 跳过偶数，不输出
        }
        cout << i << " ";
    }
    return 0;
}
// 输出: 1 3 5 7 9
```

break vs continue 对比



特性	break	continue
作用	终止整个循环	跳过本次迭代的剩余代码
对循环影响	循环不再执行，退出	循环继续，但本次循环体提前结束
适用场景	找到目标后停止搜索	忽略某些不需要处理的项
嵌套影响	只影响最内层循环（除非特殊处理）	只影响当前层循环



例题1：判断质数（使用 break 优化）



题目：输入一个正整数 n ，判断它是否为质数。可以在发现因子时立即 `break`，提高效率。

```
int main() {
    int n;
    cin >> n;
    if (n < 2) {
        cout << "No";
        return 0;
    }
    bool isPrime = true;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            isPrime = false;
            break;           // 发现因子，不用继续检查
        }
    }
}
```

“ 当 n 很大时，`break` 能显著减少循环次数。

例题2：统计非负整数（使用 continue）



题目：输入若干个整数（以 0 结束），统计其中正数的个数，忽略负数和 0。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int x, count = 0;
    while (cin >> x && x != 0) {
        if (x < 0) {
            continue;    // 负数跳过，不计数
        }
        count++;        // 只统计正数
    }
    cout << count;
    return 0;
}
```

例题3：打印 1~100 中不能被 7 整除的数



使用 `continue` 跳过能被 7 整除的数。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    for (int i = 1; i <= 100; i++) {
        if (i % 7 == 0) {
            continue;
        }
        cout << i << " ";
    }
    return 0;
}
```

嵌套循环入门



嵌套循环指一个循环内部包含另一个循环。外层循环每执行一次，内层循环会完整执行一轮。

典型应用：

- 打印二维图形（矩形、三角形）
- 遍历二维数组
- 组合问题（如乘法表）

```
for (int i = 0; i < 3; i++) {           // 外层
    for (int j = 0; j < 5; j++) {       // 内层
        cout << "*";
    }
    cout << endl;
}
// 输出 3 行, 每行 5 个 *
```

break 在嵌套循环中的行为



break 默认只跳出最内层的循环。

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 2) break;    // 只跳出内层  
        cout << i << j << " ";  
    }  
    cout << endl;  
}  
// 输出:  
// 11  
// 21  
// 31
```

“ 内层每次 $j=1$ 时输出, $j=2$ 时 `break`, 因此每行只有一个输出。 ”

如何跳出外层循环？



方法一：使用**标志变量**控制外层条件。

```
bool found = false;
for (int i = 0; i < n && !found; i++) {
    for (int j = 0; j < m; j++) {
        if (条件) {
            found = true;
            break;           // 内层 break, 外层条件也会失效
        }
    }
}
```

方法二：使用 `goto`（不推荐）或封装成函数用 `return`。

例题4：二维数组查找（跳出外层）



题目：在 3×3 的矩阵中查找是否存在数字 5，找到后输出其位置并停止所有循环。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int matrix[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    int target = 5;
    bool found = false;
    int row = -1, col = -1;

    for (int i = 0; i < 3 && !found; i++) {
        for (int j = 0; j < 3; j++) {
            if (matrix[i][j] == target) {
                row = i;
                col = j;
                found = true;
                break;          // 跳出内层
            }
        }
    }
    if (found)
        cout << "找到于 (" << row << ", " << col << ")" << endl;
    else
        cout << "未找到" << endl;
    return 0;
}
```

continue 在嵌套循环中



`continue` 也仅影响当前（最内层）循环，跳过当前迭代。

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 2) continue;  
        cout << i << j << " ";  
    }  
    cout << endl;  
}  
// 输出:  
// 11 13  
// 21 23  
// 31 33
```

“ 内层每次跳过 $j=2$ 的输出。 ”

例题5：打印下三角，跳过偶数行



题目：输入 n ，打印一个 n 行 `*` 的三角形，但偶数行（2,4,6...）不打印任何内容（使用 `continue` 跳过整个外层迭代）。

```
int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        if (i % 2 == 0) {
            cout << endl;    // 或者直接 continue，但需要换行？
            continue;        // 跳过本行打印
        }
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << endl;
    }
    return 0; // 注意：跳过外层后需要处理换行，否则输出会粘连。
}
```

常见错误与调试



错误类型	示例	解决方法
<code>break</code> 跳出多层循环失败	在嵌套循环中使用 <code>break</code> 只跳内层	使用标志变量或 <code>goto</code>
<code>continue</code> 导致死循环	<code>while</code> 中 <code>continue</code> 前未更新变量	将更新语句放在 <code>continue</code> 之前
误用 <code>continue</code> 跳过必要操作	在循环中忘记更新计数器	注意更新位置
逻辑错误	<code>break</code> 和 <code>continue</code> 条件写反	仔细检查条件

调试技巧： 在关键位置打印变量值，观察循环执行次数和跳转时机。



总结



- **break**：立即终止当前循环，常用于提前结束搜索。
- **continue**：跳过本次循环剩余部分，进入下一次迭代，常用于过滤数据。
- **嵌套循环**：**break** 和 **continue** 默认只影响最内层循环。
- 跳出外层循环需使用标志变量或 **goto**（谨慎使用）。
- 使用循环控制语句能让代码更高效、逻辑更清晰。

下节课将学习 **do-while** 循环，它与 **while** 的区别在于“至少执行一次”。



谢谢大家！



循环控制让程序更加灵活与高效！

Q & A